Name: PerimeterAlert:- Covid19 alerter by Crowdsourced, Public and Private sensors.

Members (maximum 8 members):

S. No.	Name	Role / Contribution	Contact (Optional)
1.	Madhavan Pallan	Main Author	mady902@gmail.com
2.	Family Pallan	Author	918800996753
3.			
4.			

Project Checklist

Phase 1

- ☑ ~~Project Title~~
- ☑ ~~Cover Image~~
- ☑ ~~List of Components~~
- ☑ ~~Detailed Project Description~~
- ☑ ~~Schematics~~
- ☑ ~~Code~~
- ☐ CAD/ PCB Design If any *Optional
- ☑ ~~Photo of project~~
- ☐ Video displaying working of Project
- ☐ BONUS/OPTIONAL: Video explaining working of the project

Title: PerimeterAlert:- Covid19 alerter by Crowdsourced, Public and Private sensors

Cover Image:



Video:

Component List:

ID	Name	Designator	Quantity
1	Arduino UNO / Arduino Q	MOD1	1
2	Laptop/ Android Device		1
3	5V AC-DC Power Module	MOD 2	1
4	DHT22	MOD 3	1

Project Explanation Step By Step :

We pursue the problem here as Covid19 being an air-borne infection using the medium of water droplets in the air. Water droplets in the air can be quantified and measured as humidity. It is imperative to kill the medium to cut the transfer. In this paper, we study how disturbing the medium cuts the spread. Also, we look into other parameters of weather which assist the transfer of Covid-19 including temperature and air quality index of the surrounding

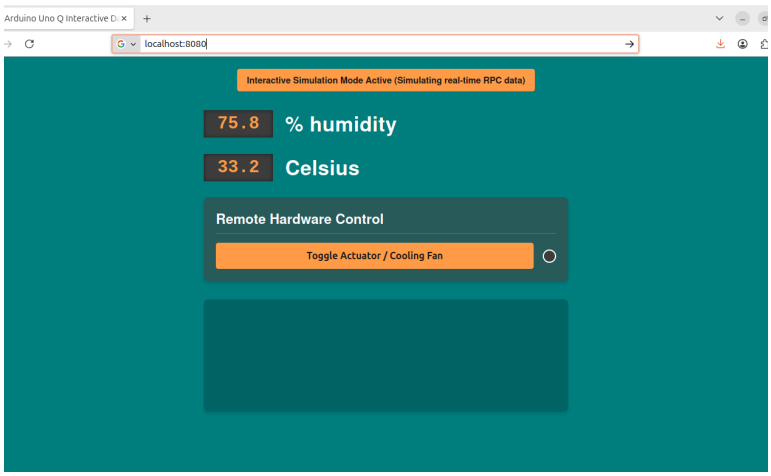


Fig 1. Author Prototype

Coding Explanation and Configuration

We follow all the requirements as required by DHT22 to interface with the MCU side of the device Arduino UNO Q followed by all the server side requirements for the IInux MPU.

Connecting a **DHT22** (temperature and humidity sensor) to the **Arduino UNO Q** is straightforward because the board features a real-time microcontroller subsystem with standard digital I/O pins (compatible with the classic UNO layout).

Step 1: Wiring Connections

Whether you are using a **DHT22 standalone sensor** (4 pins) or a **DHT22 pre-soldered module** (3 pins, which includes the pull-up resistor onboard), follow these connections:

DHT22 Pin	Function	Arduino UNO Q Connection
Pin 1 (VCC)	Power (3.3V to 5V)**5V** or **3.3V** pin	**Pin 2 (DATA)**
Single-wire data	Any digital pin (e.g., **D2**)	**Pin 3**
(NC)	Not Connected*Leave empty*	(4-pin version only)
Pin 4 (GND)	Ground	**GND** pin

Step 2: Software & Library Setup

To read data from the DHT22 on your microcontroller side:

1. Open your **Arduino IDE**.
2. Go to Sketch > Include Library > Manage Libraries..
3. Search for and install:
    - **DHT sensor library**
    - **Adafruit Unified Sensor** library (dependency required by the DHT library)

Step 3: Code

Upload the following code to the microcontroller side of your Arduino UNO Q: File name:_

```cpp
#include <DHT.h>

// Define the digital pin connected to the DHT22 data pin
#define DHTPIN 2

// Define sensor type as DHT22 (AM2302)
#define DHTTYPE DHT22

DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(9600);
  Serial.println(F("DHT22 test starting..."));

  dht.begin();
}

void loop() {
  // Wait 2 seconds between measurements (DHT22 has a 0.5Hz to 1Hz sampling rate)
  delay(2000);

  // Reading humidity and temperature
  float h = dht.readHumidity();
  float t = dht.readTemperature(); // Celsius
  float f = dht.readTemperature(true); // Fahrenheit

  // Check if readings failed and exit early to try again
  if (isnan(h) || isnan(t) || isnan(f)) {
    Serial.println(F("Failed to read from DHT sensor!"));
    return;
  }

  // Compute heat index
  float hic = dht.computeHeatIndex(t, h, false);

  // Print results to Serial Monitor
  Serial.print(F("Humidity: "));
  Serial.print(h);
  Serial.print(F("%  Temperature: "));
  Serial.print(t);
  Serial.print(F("°C ("));
  Serial.print(f);
  Serial.print(F("°F)  Heat Index: "));
  Serial.print(hic);
  Serial.println(F("°C"));
}
```

dht22.cpp

the UNO Q runs a dual-brain system (Linux MPU + Real-Time MCU), you can process these sensor readings directly on the real-time microcontroller side for fast loop response, and then use the **Arduino Bridge (RPC)** to pass the temperature and humidity metrics up to the Linux/Python side if you want to log them to a database, display them on a local web dashboard, or feed them into an edge AI script.

To host a web dashboard like the one in your screenshot on the Linux MPU side of your Arduino UNO Q, you can use **Python with Flask and Socket.IO**. This allows the Linux processor to receive sensor data from the real-time microcontroller (via the Arduino Bridge/RPC) and push it live to your browser dashboard.

Here is how to set it up step-by-step.

Step 1: Install Dependencies on the Linux MPU

SSH into your Arduino UNO Q's Linux environment and install Python, Flask, and Flask-SocketIO:

sudo apt update
sudo apt install python3-pip python3-flask python3-socketio

Verification: Run `python3 -c "import flask; import socketio; print('Installed successfully!')"` to verify installation.

Also clone/ download this repo for insight details and server templates, https://github.com/madhavanpallan/perimeteralert.git

Step 2: Create the Project Structure

Create a project folder and an `templates` folder inside it for the HTML file:

mkdir -p ~/uno_dashboard/templates

cd ~/uno_dashboard

Step 3: Create the HTML Frontend (`templates/index.html`)

Save the following file as `templates/index.html`. It matches the dark teal layout, live metrics, and toggle switch style from your screenshot

https://github.com/madhavanpallan/perimeteralert/templates/index.html

Step 4: Create the Flask Backend Script (`app.py`)

Save the following script as `app.py` in your `uno_dashboard` folder. This script initializes the server and simulates or hooks into the Arduino Bridge RPC to pull live data:

https://github.com/madhavanpallan/perimeteralert/app.py

Connection Explanation

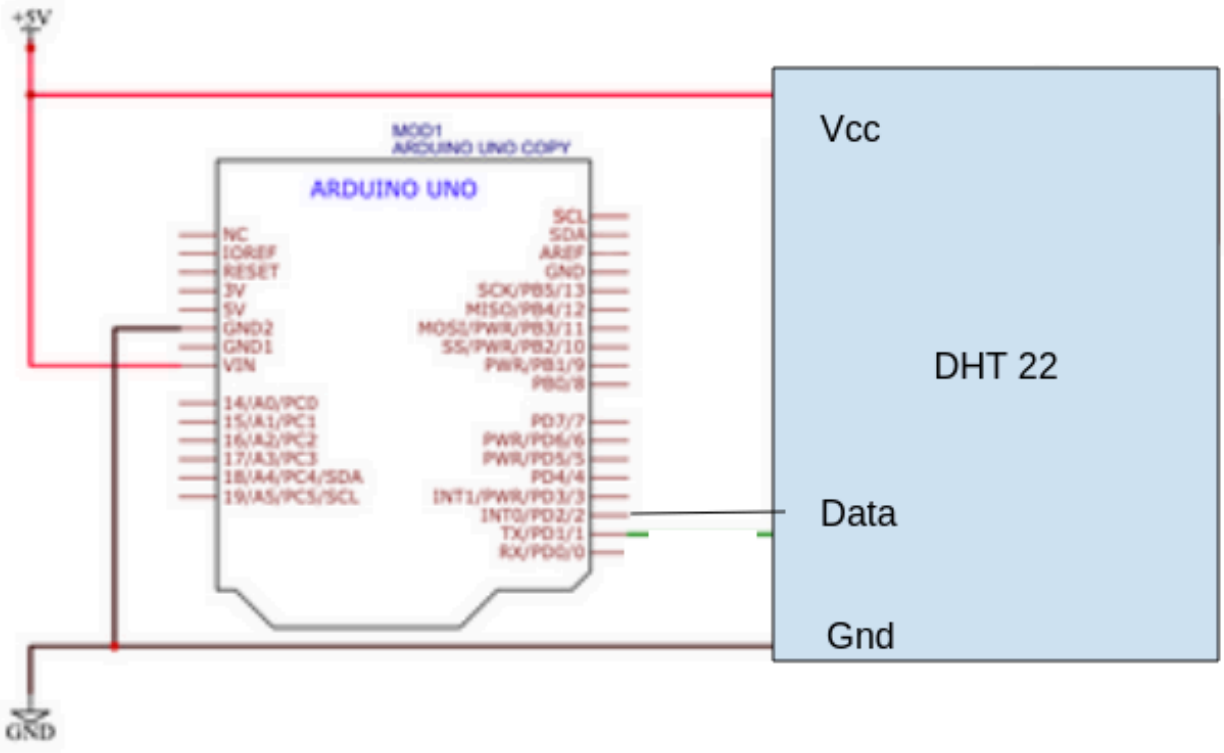


Fig 4. Circuit diagram

Testing

The DHT 22 regularly collects data from the atmosphere and stores in the MPU it can then be used for the relevant use case. The value of the temperature can be seen on the website.

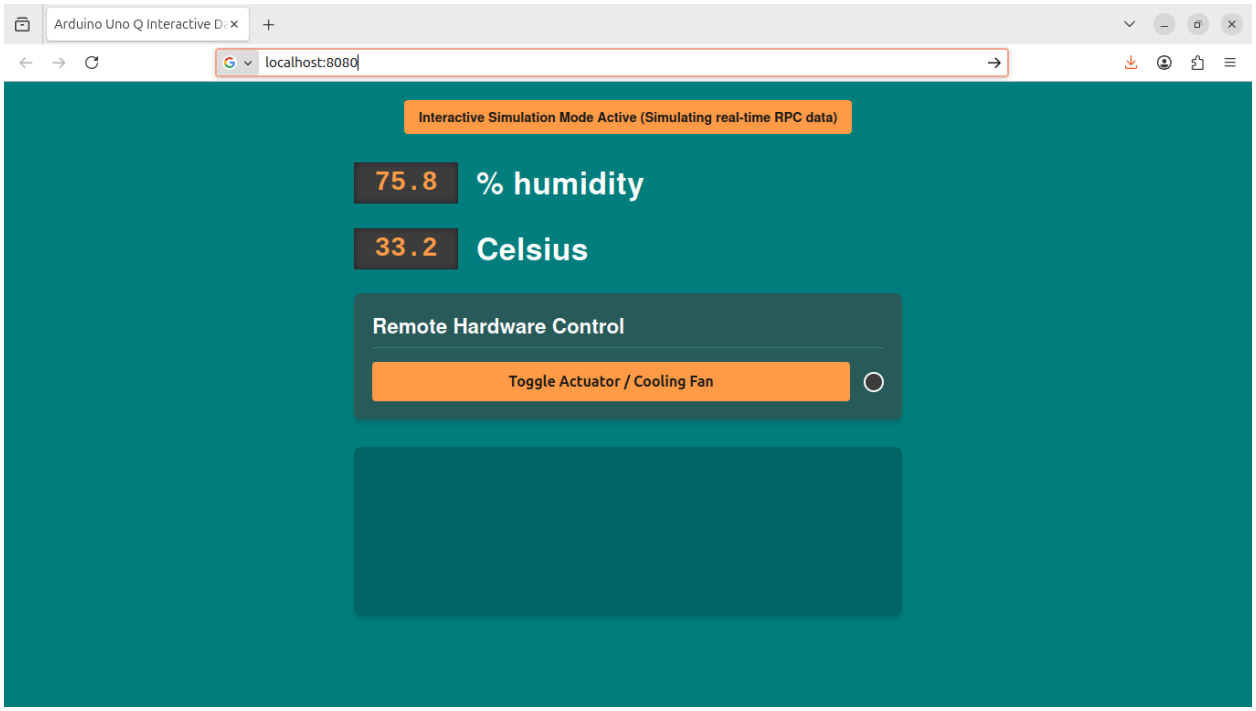


Fig 5. Testing the device with both the app interface and the touch display.

Project Video:

Given the device is programmed on a laptop, and the demo portal is available as a website, we believe the demo link is sufficient for the cause.

Future work:-

The author invites the EFY community for future collaboration and standardization of devices via his facebook page on Robotics 2009.

Code:

(Any: Github link, Drive link, Dropbox supported shared with viewer rights)
LInk:- https://github.com/madhavanpallan/perimeteralert
Research Paper:- https://drive.google.com/file/d/1joEeGnaqiyJM1Nrhw496-bYvQ5e2KLBp/view
Note:-**Madhavan Pallan has published many health related items and can be read as part of the health advisory, He is a AI Scientist and is a welcoming member.